

Gradient Boosting and XGBoost

How a chain of small corrections becomes our best single model

The problem these models solve

NOAA's ocean forecast model RTOFS reports, for any place and day, the Tropical Cyclone Heat Potential (TCHP, the heat stored above the 26 degree isotherm) and D26 (the depth of that isotherm). Autonomous Argo floats measure the same two quantities at scattered points. Comparing the two at the same place and day shows the forecast is biased. We therefore learn a correction: a function that predicts the forecast's error from information available at forecast time, so it can be subtracted everywhere.

Formally, we have n observation pairs. For pair i , let x_i be a vector of d known quantities (position, calendar, and fields taken from the forecast itself) and let

$$\delta_i = y_i^{\text{Argo}} - y_i^{\text{RTOFS}}$$

be the observed error of the forecast. We seek a function f minimizing the expected size of $\delta - f(x)$, and the corrected forecast is then

$$\hat{y}(x) = y^{\text{RTOFS}}(x) + f(x).$$

All models below are different ways of constructing f from the n samples. Their quality is measured by the mean absolute error of the corrected forecast on dates that come after every date used to fit f , so the score always reflects prediction of the future, never memory of the past.

1. Fitting a sum of small corrections

Gradient boosting builds f as a sum of M small trees, fitted one after another, each correcting what the sum so far still gets wrong:

$$F_M(x) = \sum_{m=1}^M \nu f_m(x), \quad 0 < \nu \leq 1.$$

With squared-error loss the recipe is simple. Let $r_i = \delta_i - F_{m-1}(x_i)$ be the residuals of the current sum. Fit the next small tree f_m to the residuals, add it in, and repeat. The factor ν (the learning rate) deliberately takes only a small step in the direction of each new tree; many small steps generalize better than few large ones.

The name comes from viewing this as gradient descent in function space: for a general loss ℓ , the residuals are replaced by the negative gradient $-\partial\ell(\delta_i, F(x_i))/\partial F(x_i)$, so each tree is a descent step.

2. What XGBoost adds

XGBoost is an implementation of this idea that chooses each tree using a second-order expansion of the loss plus an explicit penalty on tree complexity. For a fixed tree structure, let G

and H be the sums of first and second derivatives of the loss over the samples in a leaf. The optimal leaf value and the value of a candidate split have closed forms:

$$w^* = -\frac{G}{H + \lambda}, \quad \text{gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right],$$

where λ is a regularization constant that shrinks leaf values toward zero. Splits are only kept if their gain is positive after the penalty, which is how the method resists fitting noise. Two further randomizations mirror the forest: each tree sees a random 80 percent of the samples and of the input coordinates.

The contrast with a random forest is the key point. A forest averages many deep, independent trees fitted in parallel to the same target. Boosting chains many shallow trees, each fitted to what remains unexplained. Shallow trees are weak alone but the chain is expressive, and because every tree works on the current residual, redundant inputs cost little: once one of two near-duplicate inputs has been used, the residual no longer rewards using the other.

3. How we apply it to the RTOFS correction

Our locked setting, fixed early and never tuned per experiment: $M = 300$ trees of depth 4, learning rate 0.03, 80 percent row and column sampling, $\lambda = 1$. Missing input values are replaced by the median of the fitting period.

- This is the engine of the recommended model. As a single global model it reaches 11.40 (TCHP) and 10.76 m (D26) against 16.6 / 14.9 for the raw forecast.
- It is also the expert inside the mixture-of-experts blend (see the mixture primer), which reaches 11.19 / 10.55.
- A caution from the position-only experiment: with only latitude and longitude as inputs, depth-4 trees cannot carve fine spatial boxes, and boosting was the worst of the four families there. Its strength appears only when many informative inputs give the shallow trees good one-variable questions to ask.

Where it is strong and weak here

Strong: the best accuracy of every family on the full input set, graceful with redundant inputs, fast. Weak: needs its handful of settings chosen sensibly, is as discontinuous as any tree method, and like all tree methods it cannot predict values outside the range of the targets it was fitted on.